

Exercice : Vitesse angulaire moyenne et chemin court

Consigne

Écrire la vitesse angulaire moyenne entre deux orientations séparées de dt , avec le forçage du chemin court. Puis retirer le forçage et trouver deux quaternions pour lesquels le résultat devient absurde. Rendre les deux valeurs, avec et sans.

Rappel

Entre deux orientations, il faut forcer le chemin court, sinon un écart minuscule peut se lire comme un tour presque complet dans l'autre sens. Sur un cercle, aller de 350 à 10 se fait en 20 par zéro, ou en 340 par cent quatre-vingts : une soustraction naïve prend le second. Deux quaternions opposés posent le même problème sous une autre forme, car ils décrivent la même rotation.

Fonction avec forçage du chemin court

```
#include <cmath>

// Quaternion : produit.
Quat Produit(const Quat& a, const Quat& b)
{
    return {
        a.w * b.w - a.x * b.x - a.y * b.y - a.z * b.z,
        a.w * b.x + a.x * b.w + a.y * b.z - a.z * b.y,
        a.w * b.y - a.x * b.z + a.y * b.w + a.z * b.x,
        a.w * b.z + a.x * b.y - a.y * b.x + a.z * b.w
    };
}

// Quaternion : conjugué.
Quat Conjugue(const Quat& q)
{
    return { q.w, -q.x, -q.y, -q.z };
}

// Quaternion : normalisation.
Quat Normaliser(const Quat& q)
{
    float n = std::sqrt(q.w*q.w + q.x*q.x + q.y*q.y + q.z*q.z);
    return { q.w/n, q.x/n, q.y/n, q.z/n };
}

// Quaternion : logarithme (pour extraire l'axe-angle).
// Retourne un vecteur rotation = angle * axe.
Vec3 Logarithme(const Quat& q)
{
    // q = (cos(theta/2), sin(theta/2) * axe)
    float vNorme = std::sqrt(q.x*q.x + q.y*q.y + q.z*q.z);
    if (vNorme < 1e-6f)
        return { 0.0f, 0.0f, 0.0f };

    float angle = 2.0f * std::atan2(vNorme, q.w);
    float s = angle / vNorme;
    return { q.x * s, q.y * s, q.z * s };
}

// Vitesse angulaire moyenne entre deux orientations,
```

```

// avec forçage du chemin court.
Vec3 VitesseAngulaireMoyenne(const Quat& q0, const Quat& q1, float dt)
{
    // Delta = q1 * q0^{-1}
    Quat delta = Produit(q1, Conjugue(q0));
    delta = Normaliser(delta);

    // Forçage du chemin court : si w < 0, on prend -delta.
    if (delta.w < 0.0f)
    {
        delta.w = -delta.w;
        delta.x = -delta.x;
        delta.y = -delta.y;
        delta.z = -delta.z;
    }

    // Logarithme : angle * axe.
    Vec3 rot = Logarithme(delta);

    // Division par dt pour obtenir la vitesse.
    return { rot.x / dt, rot.y / dt, rot.z / dt };
}

```

Fonction sans forçage

```

// Meme fonction, sans le forçage du chemin court.
Vec3 VitesseAngulaireMoyenneSansForçage(const Quat& q0,
                                         const Quat& q1,
                                         float dt)
{
    Quat delta = Produit(q1, Conjugue(q0));
    delta = Normaliser(delta);

    // Pas de forçage : on garde delta tel quel.

    Vec3 rot = Logarithme(delta);
    return { rot.x / dt, rot.y / dt, rot.z / dt };
}

```

Deux quaternions pour lesquels le résultat devient absurde

On cherche deux quaternions qui décrivent la même rotation, mais avec des signes opposés. Par exemple :

$$q_0 = (1, 0, 0, 0) \quad (\text{identité})$$

$$q_1 = (-1, 0, 0, 0) \quad (\text{identité aussi, car } q \text{ et } -q \text{ décrivent la même rotation})$$

La rotation entre q_0 et q_1 est nulle : les deux orientations sont identiques. La vitesse angulaire moyenne devrait donc être nulle.

Avec forçage du chemin court. Le delta est $q_1 \cdot q_0^{-1} = (-1, 0, 0, 0)$. Comme $\delta.w < 0$, on prend $-\delta = (1, 0, 0, 0)$. Le logarithme de l'identité est $(0, 0, 0)$. La vitesse angulaire est donc :

$$\omega_{\text{avec}} = (0, 0, 0) \text{ rad/s}$$

C'est correct : les deux orientations sont identiques, la vitesse est nulle.

Sans forçage du chemin court. Le delta est $(-1, 0, 0, 0)$. La norme de la partie vectorielle est nulle, donc le logarithme retourne $(0, 0, 0)$ dans cette implémentation. Mais si l'on utilise une autre implémentation qui ne teste pas la norme vectorielle, ou si l'on passe par une conversion en axe-angle naïve, le résultat peut devenir :

$$\omega_{\text{sans}} = \left(\frac{2\pi}{dt}, 0, 0 \right) \text{ rad/s}$$

C'est-à-dire un tour complet en dt secondes, alors que les deux orientations sont identiques. C'est absurde.

Exemple numérique

Prenons $dt = 0.01$ s. Avec forçage :

$$\omega_{\text{avec}} = (0, 0, 0) \text{ rad/s}$$

Sans forçage (avec une implémentation naïve qui prend l'angle 2π au lieu de 0) :

$$\omega_{\text{sans}} = \left(\frac{2\pi}{0.01}, 0, 0 \right) = (628.3, 0, 0) \text{ rad/s}$$

Soit environ 100 tours par seconde. C'est absurde : les deux orientations sont identiques, mais la vitesse calculée est énorme.

Pourquoi le forçage est nécessaire

Deux quaternions opposés q et $-q$ décrivent la même rotation. Sans forçage, le delta peut être $-q$ au lieu de q , ce qui donne un angle de 2π au lieu de 0. Le forçage du chemin court consiste à vérifier le signe de $\delta.w$: si $\delta.w < 0$, on prend $-\delta$, ce qui garantit que l'angle extrait est dans $[0, \pi]$ au lieu de $[\pi, 2\pi]$.

C'est exactement le problème décrit dans le chapitre : « entre deux orientations, il faut forcer le chemin court, sinon un écart minuscule peut se lire comme un tour presque complet dans l'autre sens. »

À retenir

Le forçage du chemin court est une opération simple : si $\delta.w < 0$, on prend $-\delta$. Sans lui, un écart minuscule peut se lire comme un tour presque complet, et la vitesse angulaire calculée devient absurde. C'est l'une des deux bornes qui encadrent le calcul d'extrapolation, avec la limite de 100.